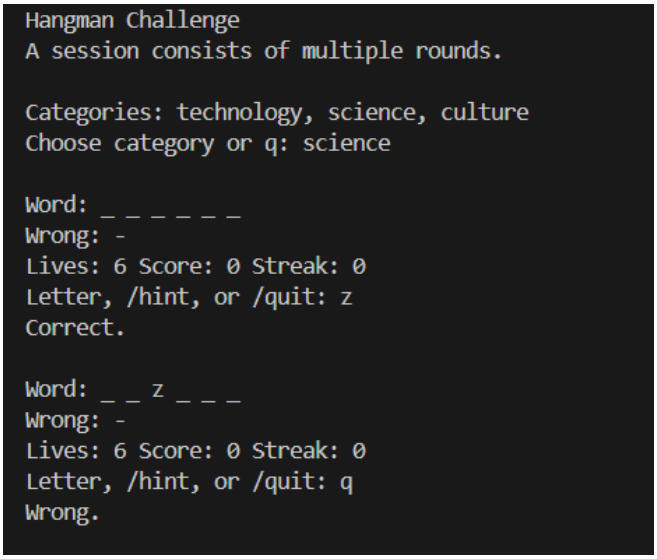
SE LAB LAB-4
NAME:VARUN SAHU
SRN:PES2UG24CS576
SECTION:J

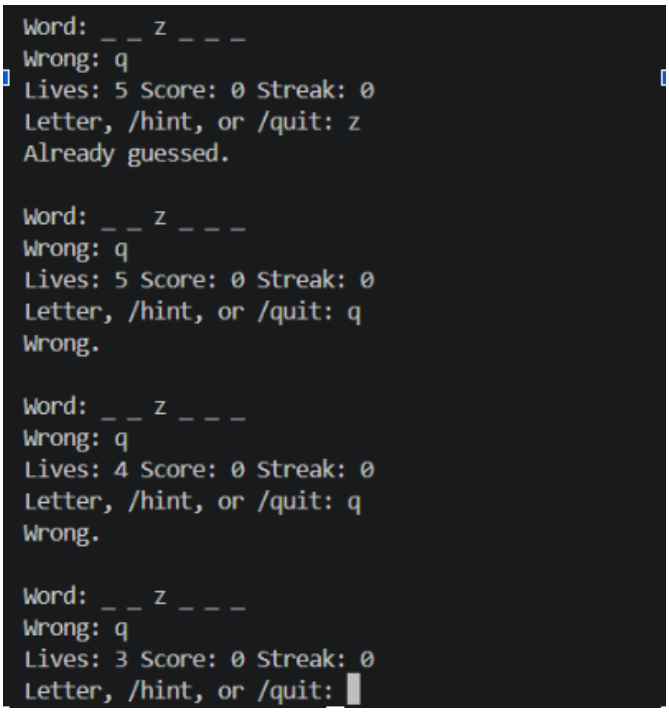
Objective
Use Vibe Coding tools to fix the broken code and add features to the same. Learn how to prompt to get the job done in under three to four attempts. Clone a repo assigned to you, make changes and push to your repo in your personal account.

Repo: https://github.com/varunsahu5002/SE_ARCHITECTURE_LAB4.git

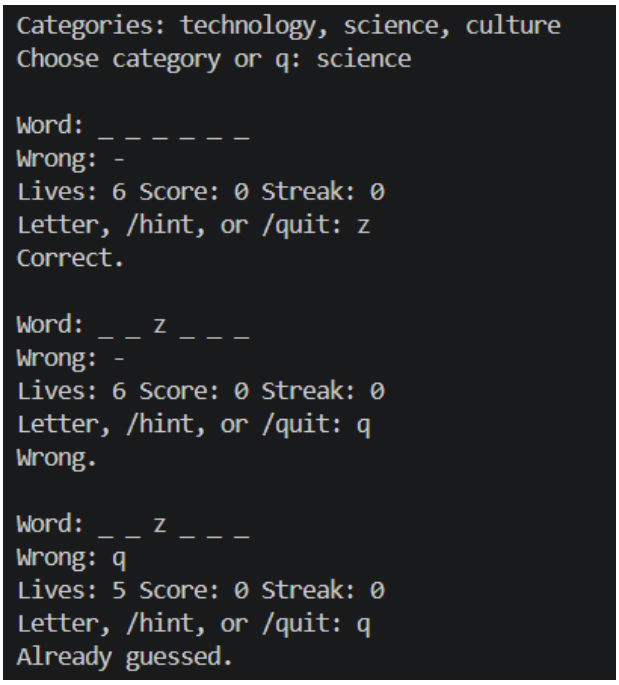
Task 1 — Guess-state correctness

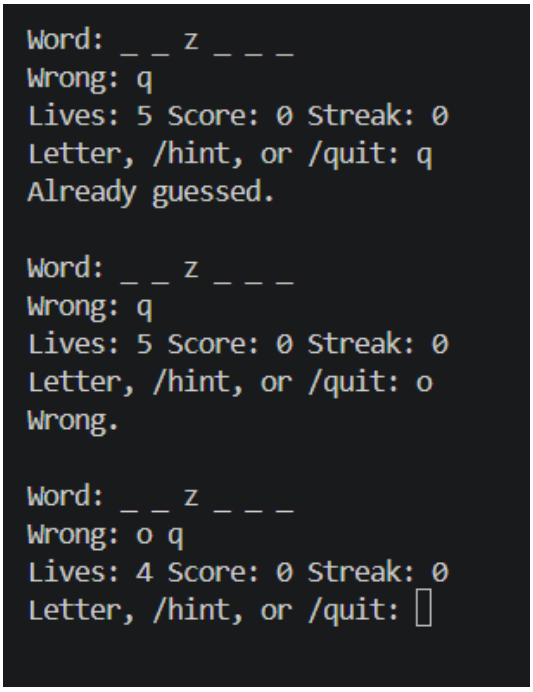
Correct repeated-guess handling so a previously attempted wrong letter cannot consume another life, and a previously accepted correct letter cannot be counted as a new guess.
Done when: each distinct letter affects the round exactly once.
Before:





After:



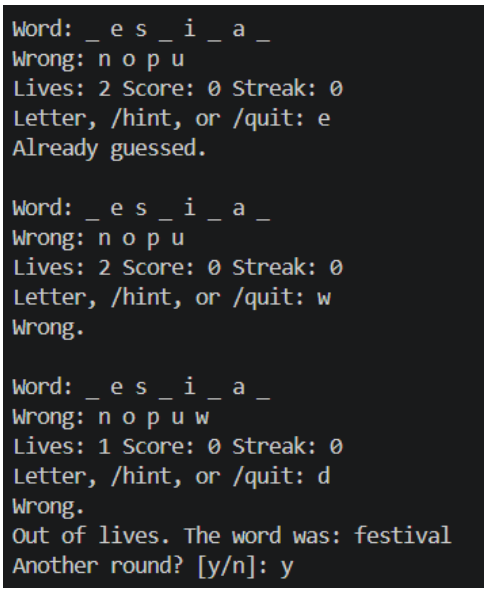


Task 2 — Complete the session model

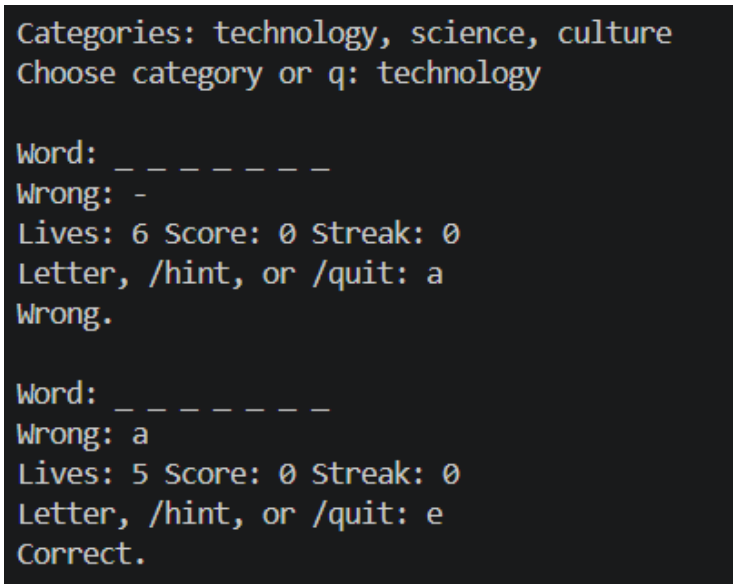
Integrate the supplied session-statistics component so rounds played, rounds won, and best streak are tracked correctly across multiple rounds.
Done when: starting a new round resets only round-specific state; session statistics continue across the whole run.

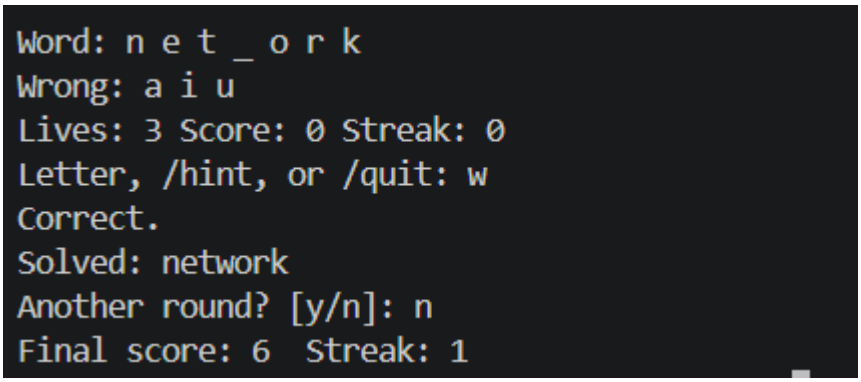
Before:

Round 1



round 2:

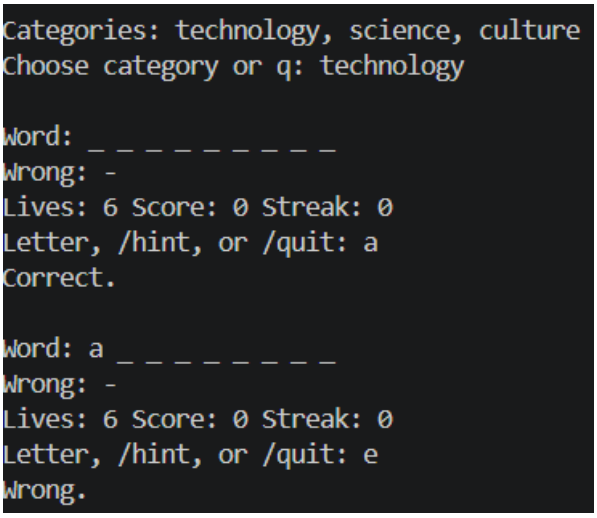




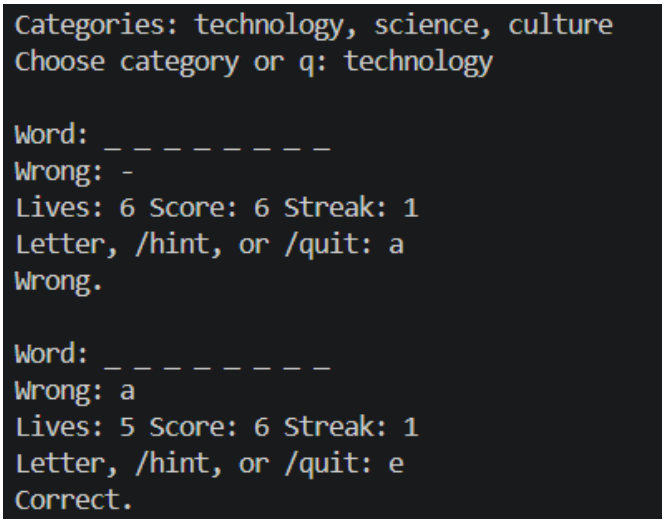
Bug here is not displaying match stats.

After:

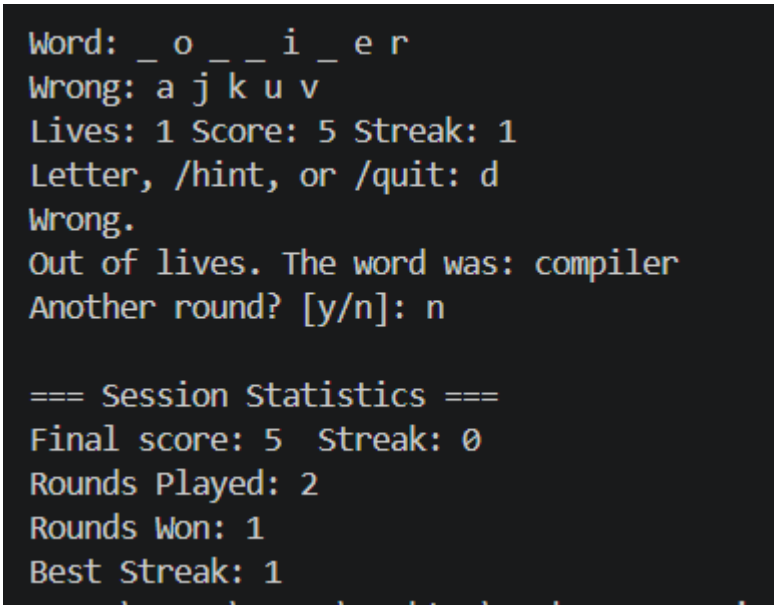
Round1:



round 2:



Bug fix:



The match stats have been displayed by making changes to [game.py]

Task 3 — Difficulty and scoring

Add difficulty choices that alter the available lives and scoring. Preserve category selection and make hint usage affect scoring consistently.
Done when: difficulty changes round rules without leaking state between rounds.

Before:
The game hardcodes your starting lives to 6.
It never asks you to choose a difficulty.
It always subtracts exactly 1 point when you use a hint, and the points you earn for winning never scale.

```
Categories: technology, science, culture
Choose category or q: culture

Word: _ _ _ _ _ _ _ _ _
Wrong: -
Lives: 6 Score: 13 Streak: 2
Letter, /hint, or /quit: a
Correct.

Word: _ a _ _ _ a _ _
Wrong: -
Lives: 6 Score: 13 Streak: 2
Letter, /hint, or /quit: e
Correct.
```

```
Word: _ a _ _ u a _ e
Wrong: i o
Lives: 4 Score: 13 Streak: 2
Letter, /hint, or /quit: /hint
A system used for communication.

Word: _ a _ _ u a _ e
Wrong: i o
Lives: 4 Score: 12 Streak: 2
Letter, /hint, or /quit: l
Correct.
```

After:
Difficulty can be chosen
When easy is chosen lives are 8(no hardcode
Of 6 lives)
when difficulty: difficult is selected upon using /hint
Score drops by 3 instead of 1
Lives start with 4 instead of hardcoded 6 lives

```
Categories: technology, science, culture
Choose category or q: technology
Choose difficulty (easy/medium/hard): easy

Word: _ _ _ _ _ _ _ _
Wrong: -
Lives: 8 Score: 0 Streak: 0
Letter, /hint, or /quit: a
Wrong.
```

```
Word: _ e _ _ _ a _ e
Wrong: -
Lives: 4 Score: 6 Streak: 1
Letter, /hint, or /quit: d
Wrong.

Word: _ e _ _ _ a _ e
Wrong: d
Lives: 3 Score: 6 Streak: 1
Letter, /hint, or /quit: a
Already guessed.

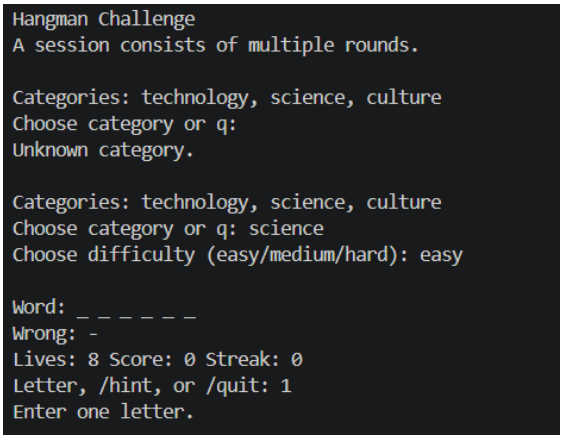
Word: _ e _ _ _ a _ e
Wrong: d
Lives: 3 Score: 6 Streak: 1
Letter, /hint, or /quit: /hint
Something inherited from previous generations.

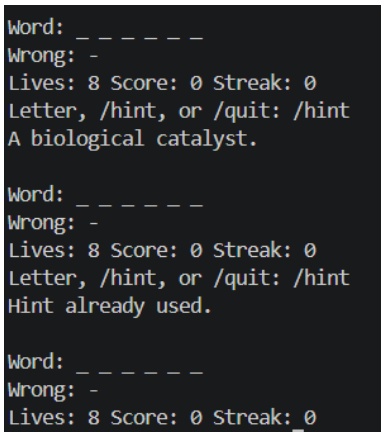
Word: _ e _ _ _ a _ e
Wrong: d
Lives: 3 Score: 3 Streak: 1
Letter, /hint, or /quit: h
Correct.
```

Task 4 — Robust input and feedback

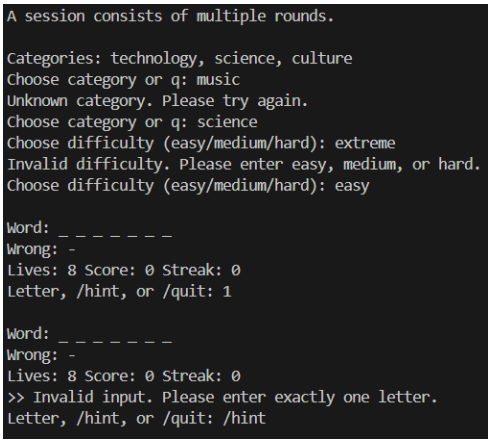
Improve command handling for invalid letters, repeated commands, category selection, and hint usage. Feedback should describe actual player actions once.
Done when: malformed input never changes game state and feedback is not duplicated.

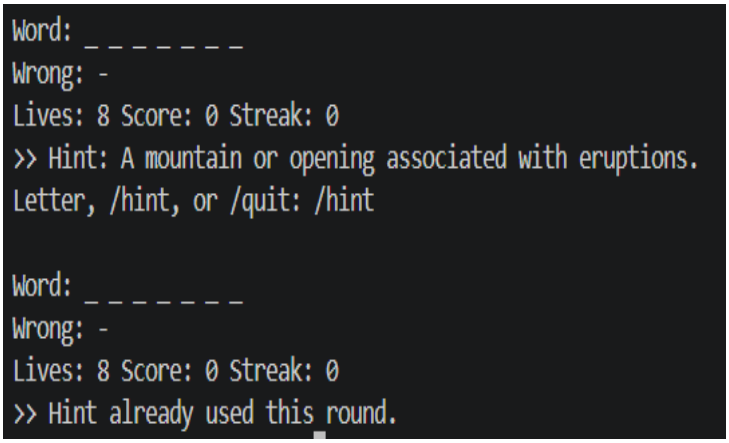
Before:
Invalid Category Handling: Typing an incorrect category forces the entire main menu to reprint instead of simply asking for the input again.
Buried Feedback: Error messages (like "Enter one letter") are pushed out of sight because the game immediately reprints the full board right below them.
Unprotected Inputs: Hitting enter on an empty prompt or typing symbols causes the game to cycle pointlessly, cluttering the terminal history.





After:
Clean Menus: Invalid category or difficulty choices now smoothly ask for the input again without reprinting the entire menu.
Targeted Feedback: Error messages are held and displayed clearly right above the next input prompt (using `>>`) instead of getting buried by the board.
Strict Validation: The game properly catches empty inputs, numbers, and symbols, preventing the board from pointlessly cycling.





game.py:

```
import random
from words import WORDS, HINTS
from stats import SessionStats

class HangmanGame:
    def __init__(self):
        self.score = 0
        self.streak = 0
        self.stats = SessionStats()
        self.category = "technology"
        self.secret = ""
        self.guessed_letters = set()
        self.wrong_letters = set()
        self.lives = 6
        self.hint_used = False

        self.current_difficulty = "medium"
        self.difficulty_lives = {"easy": 8, "medium": 6, "hard": 4}
        self.difficulty_mult = {"easy": 1, "medium": 2, "hard": 3}

    def start_round(self):
        self.secret = random.choice(WORDS[self.category])
        self.guessed_letters.clear()
        self.wrong_letters.clear()
        self.lives = self.difficulty_lives[self.current_difficulty]
        self.hint_used = False

    def masked(self):
        return " ".join(ch if ch in self.guessed_letters else "_" for ch in self.secret)

    def won(self):
        return all(ch in self.guessed_letters for ch in set(self.secret))

    def guess(self, letter):
        # Task 4: Robust input handling and clear feedback
        if len(letter) != 1 or not letter.isalpha():
            return "Invalid input. Please enter exactly one letter."

        if letter in self.guessed_letters:
            return f"You already guessed '{letter}'."

        self.guessed_letters.add(letter)

        if letter in self.secret:
            return "Correct guess!"
```

```
        self.wrong_letters.add(letter)
        self.lives -= 1
        return "Wrong guess."

    def use_hint(self):
        if self.hint_used:
            return "Hint already used this round."
        self.hint_used = True
        penalty = 1 * self.difficulty_mult[self.current_difficulty]
        self.score = max(0, self.score - penalty)
        return f"Hint: {HINTS.get(self.secret, 'No hint available.')}"

    def play_round(self):
        self.start_round()
        feedback = ""

        while self.lives > 0 and not self.won():
            print("\nWord:", self.masked())
            print("Wrong:", " ".join(sorted(self.wrong_letters)) or "-")
            print("Lives:", self.lives, "Score:", self.score, "Streak:", self.streak)

            # Task 4: Print feedback contextually before the next input
            if feedback:
                print(">>", feedback)
                feedback = ""

            raw = input("Letter, /hint, or /quit: ").strip().lower()

            if raw == "/quit":
                return False
            if raw == "/hint":
                if self.hint_used:
                    feedback = "Hint already used this round."
                else:
                    feedback = self.use_hint()
                continue

            feedback = self.guess(raw)

        if self.won():
            self.streak += 1
            round_score = (5 + self.streak) * self.difficulty_mult[self.current_difficulty]
            self.score += round_score
            print("\nSolved:", self.secret)
            print(f"Earned {round_score} points! Total Score: {self.score}")
            self.stats.record(True, self.streak)
            return True
```

```
        self.streak = 0
        print("\nOut of lives. The word was:", self.secret)
        self.stats.record(False, self.streak)
        return True

    def run(self):
        print("Hangman Challenge")
        print("A session consists of multiple rounds.")
        while True:
            print("\nCategories:", ", ".join(WORDS))

            # Task 4: Robust category selection loop
            while True:
                raw = input("Choose category or q: ").strip().lower()
                if raw == "q" or raw in WORDS:
                    break
                print("Unknown category. Please try again.")

            if raw == "q":
                break

            self.category = raw

            # Task 4: Robust difficulty selection loop
            while True:
                diff = input("Choose difficulty (easy/medium/hard): ").strip().lower()
                if diff in self.difficulty_lives:
                    self.current_difficulty = diff
                    break
                print("Invalid difficulty. Please enter easy, medium, or hard.")

            if not self.play_round():
                break

            again = input("Another round? [y/n]: ").strip().lower()
            if again != "y":
                break

        print("\n=== Session Statistics ===")
        print("Final score:", self.score, " Streak:", self.streak)
        print("Rounds Played:", self.stats.rounds)
        print("Rounds Won:", self.stats.wins)
        print("Best Streak:", self.stats.best_streak)
```

main.py:

```
from game import HangmanGame
```

```
if __name__ == "__main__":
    HangmanGame().run()
```

stats.py:

```
class SessionStats:
    def __init__(self):
        self.rounds = 0
        self.wins = 0
        self.best_streak = 0

    def record(self, won, streak):
        self.rounds += 1
        self.wins += int(won)
        self.best_streak = max(self.best_streak, streak)
```

words.py:

```
WORDS = {
    "technology": ["python", "compiler", "database", "algorithm", "network"],
    "science": ["gravity", "neuron", "galaxy", "enzyme", "volcano"],
    "culture": ["festival", "painting", "theatre", "heritage", "language"],
}

HINTS = {
    "python": "A programming language named after a snake.",
    "compiler": "Turns source code into another form.",
    "database": "Stores structured information.",
    "algorithm": "A step-by-step method for solving a problem.",
    "network": "A connected collection of devices or systems.",
    "gravity": "A force that attracts masses.",
    "neuron": "A cell that carries signals in the nervous system.",
    "galaxy": "A huge system of stars.",
    "enzyme": "A biological catalyst.",
    "volcano": "A mountain or opening associated with eruptions.",
    "festival": "A celebration or series of events.",
    "painting": "A visual artwork made with pigments.",
    "theatre": "A place or art form involving staged performance.",
    "heritage": "Something inherited from previous generations.",
    "language": "A system used for communication.",
}
```